





Jai-Kumar786 /
Assignments







<> Code

 Issues

 Pull requests

 Actions

 Projects

 Wiki

 Security





 master ▾

Assignments / A123 battery / low ocv curr -10.ipynb





 Jai-Kumar786

05-18-2025 Analysed data on A123 Battery

0f5b7e0 · yesterday



9042 lines (9042 loc) · 814 KB

Preview

Code

Blame

Raw









LOW CURRENT OCV AT -10 degree celcius

In [37]:

```
import pandas as pd

# Read a specific sheet by name
low_curr_ocv_minus_10= pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assig

low_curr_ocv_minus_10.head(10)
```

Out[37]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.010944	2012-06-29 16:51:38	3.010944	1	1	0.
1	2	3.026961	2012-06-29 16:51:38	0.015765	2	1	1.
2	3	8.035992	2012-06-29 16:51:44	5.007575	3	1	0.
3	4	13.043648	2012-06-29 16:51:49	10.015231	3	1	0.
4	5	18.051127	2012-06-29 16:51:54	15.022710	3	1	0.
5	6	23.058702	2012-06-29 16:51:59	20.030285	3	1	0.
6	7	28.066285	2012-06-29 16:52:04	25.037868	3	1	0.
7	8	33.073845	2012-06-29 16:52:09	30.045427	3	1	0.
8	9	38.081417	2012-06-29 16:52:14	35.053000	3	1	0.
9	10	43.089009	2012-06-29 16:52:19	40.060591	3	1	0.

In [38]:

```
low_curr_ocv_minus_10.describe()
```

Out[38]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	29785.00000	29785.000000	29785	29785.000000	29785.000000
mean	14893.00000	79452.354943	2012-06-30 14:55:48.690951936	36718.816502	5.960987
min	1.00000	3.010944	2012-06-29 16:51:38	0.015765	1.000000
25%	7447.00000	40580.632806	2012-06-30 04:07:57	18022.251663	5.000000
50%	14893.00000	77867.013967	2012-06-30 14:29:23	36665.442255	5.000000
75%	22339.00000	118452.093694	2012-07-01 01:45:49	55308.632827	7.000000
max	29785.00000	159037.139119	2012-07-01 13:02:14	76334.569682	8.000000
std	8598.33322	44515.946548	NaN	21570.964248	1.013849

In [39]:

```
low_curr_ocv_minus_10.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29785 entries, 0 to 29784
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            29785 non-null  int64
1   Test_Time(s)                          29785 non-null  float64
2   Date_Time                             29785 non-null  datetime64[ns]
3   Step_Time(s)                          29785 non-null  float64
4   Step_Index                            29785 non-null  int64
5   Cycle_Index                           29785 non-null  int64
6   Current(A)                            29785 non-null  float64
7   Voltage(V)                            29785 non-null  float64
8   Charge_Capacity(Ah)                   29785 non-null  float64
9   Discharge_Capacity(Ah)                 29785 non-null  float64
10  Charge_Energy(Wh)                      29785 non-null  float64
11  Discharge_Energy(Wh)                   29785 non-null  float64
12  dV/dt(V/s)                            29785 non-null  float64
13  Internal_Resistance(Ohm)                29785 non-null  int64
14  Is_FC_Data                             29785 non-null  int64
15  AC_Impedance(Ohm)                      29785 non-null  int64
16  ACI_Phase_Angle(Deg)                   29785 non-null  int64
17  Temperature (C)_1                       29785 non-null  float64
18  Temperature (C)_2                       29785 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.3 MB
```

LOW CURRENT OCV AT 0 degree celcius

```
low_curr_ocv_0=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A
```

```
In [41]: low_curr_ocv_0.head(10)
```

Out[41]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.014942	2012-06-18 11:49:18	3.014942	1	1	0.
1	2	3.046301	2012-06-18 11:49:18	0.031089	2	1	1.
2	3	8.054232	2012-06-18 11:49:23	5.007584	3	1	0.
3	4	13.061941	2012-06-18 11:49:28	10.015293	3	1	0.
4	5	18.073395	2012-06-18 11:49:33	15.026747	3	1	0.
5	6	23.080863	2012-06-18 11:49:38	20.034215	3	1	0.
6	7	28.088504	2012-06-18 11:49:43	25.041856	3	1	0.
7	8	33.096150	2012-06-18 11:49:48	30.049501	3	1	0.
8	9	38.103670	2012-06-18 11:49:53	35.057022	3	1	0.
9	10	43.111231	2012-06-18 11:49:58	40.064583	3	1	0.

```
In [42]: low_curr_ocv_0.describe()
```

Out[42]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	30249.000000	30249.000000	30249	30249.000000	30249.000000
mean	15125.000000	80615.643879	2012-06-19 10:12:51.466792448	36962.274515	5.962048
min	1.000000	3.014942	2012-06-18 11:49:18	0.031089	1.000000

25%	7563.000000	41162.668955	2012-06-18 23:15:18	17997.213788	5.000000
50%	15125.000000	79029.928468	2012-06-19 09:46:26	36930.843545	5.000000
75%	22687.000000	120193.073943	2012-06-19 21:12:29	55864.473308	7.000000
max	30249.000000	161359.267642	2012-06-20 08:38:36	76186.533244	8.000000
std	8732.278483	45209.697396	NaN	21853.017650	1.030102



In [43]: `low_curr_ocv_0.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30249 entries, 0 to 30248
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            30249 non-null  int64
1   Test_Time(s)                          30249 non-null  float64
2   Date_Time                             30249 non-null  datetime64[ns]
3   Step_Time(s)                          30249 non-null  float64
4   Step_Index                            30249 non-null  int64
5   Cycle_Index                           30249 non-null  int64
6   Current(A)                           30249 non-null  float64
7   Voltage(V)                           30249 non-null  float64
8   Charge_Capacity(Ah)                  30249 non-null  float64
9   Discharge_Capacity(Ah)               30249 non-null  float64
10  Charge_Energy(Wh)                    30249 non-null  float64
11  Discharge_Energy(Wh)                 30249 non-null  float64
12  dV/dt(V/s)                           30249 non-null  float64
13  Internal_Resistance(Ohm)              30249 non-null  int64
14  Is_FC_Data                           30249 non-null  int64
15  AC_Impedance(Ohm)                    30249 non-null  int64
16  ACI_Phase_Angle(Deg)                  30249 non-null  int64
17  Temperature (C)_1                     30249 non-null  float64
18  Temperature (C)_2                     30249 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.4 MB
```

LOW CURRENT OCV AT 10 degree celcius

In [44]: `low_curr_ocv_10=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\`



In [45]: `low_curr_ocv_10.head(10)`

Out[45]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
			2012-06-				
0	1	3.014079	11	3.014079	1	1	0

			18:47:47			
1	2	3.025053	2012-06-11 18:47:47	0.000002	2	1 1.
2	3	8.032849	2012-06-11 18:47:52	5.007554	3	1 0.
3	4	13.040499	2012-06-11 18:47:57	10.015204	3	1 0.
4	5	18.046321	2012-06-11 18:48:02	15.021026	3	1 0.
5	6	23.053825	2012-06-11 18:48:07	20.028530	3	1 0.
6	7	28.061527	2012-06-11 18:48:12	25.036233	3	1 0.
7	8	33.068989	2012-06-11 18:48:17	30.043694	3	1 0.
8	9	38.076571	2012-06-11 18:48:22	35.051276	3	1 0.
9	10	43.084085	2012-06-11 18:48:27	40.058790	3	1 0.



In [46]:

low_curr_ocv_10.describe()

Out[46]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	31898.00000	31898.000000	31898	31898.000000	31898.000000
mean	15949.50000	84407.542774	2012-06-12 23:21:28.604269824	35034.642104	5.807605
min	1.00000	3.014079	2012-06-11 18:47:47	0.000002	1.000000
25%	7975.25000	43127.482188	2012-06-12 12:56:42.249999872	14612.052933	5.000000
50%	15949.50000	83059.113340	2012-06-13 00:02:14.500000	34527.490563	5.000000
75%	23923.75000	126287.349589	2012-06-13 12:02:42.750000128	54492.820953	7.000000
max	31898.00000	169513.642825	2012-06-14 00:03:10	75041.088878	8.000000

std

9208.30378

47856.153587

NaN

22318.590732

1.210257

In [47]:

low_curr_ocv_10.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31898 entries, 0 to 31897
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            31898 non-null  int64
1   Test_Time(s)                          31898 non-null  float64
2   Date_Time                             31898 non-null  datetime64[ns]
3   Step_Time(s)                          31898 non-null  float64
4   Step_Index                            31898 non-null  int64
5   Cycle_Index                           31898 non-null  int64
6   Current(A)                            31898 non-null  float64
7   Voltage(V)                            31898 non-null  float64
8   Charge_Capacity(Ah)                   31898 non-null  float64
9   Discharge_Capacity(Ah)                31898 non-null  float64
10  Charge_Energy(Wh)                     31898 non-null  float64
11  Discharge_Energy(Wh)                  31898 non-null  float64
12  dV/dt(V/s)                            31898 non-null  float64
13  Internal_Resistance(Ohm)               31898 non-null  int64
14  Is_FC_Data                            31898 non-null  int64
15  AC_Impedance(Ohm)                     31898 non-null  int64
16  ACI_Phase_Angle(Deg)                  31898 non-null  int64
17  Temperature (C)_1                     31898 non-null  float64
18  Temperature (C)_2                     31898 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.6 MB
```

LOW CURRENT OCV AT 20 degree celcius

In [48]:

low_curr_ocv_20=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\

In [49]:

low_curr_ocv_20.head(10)

Out[49]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.007810	2012-06-14 12:13:15	3.007810	1	1	0.
1	2	3.008080	2012-06-14 12:13:15	0.000002	2	1	1.
2	3	8.015990	2012-06-14 12:13:20	5.007662	3	1	0.
3	4	13.023481	2012-06-14 12:13:25	10.015153	3	1	0.

			12:13:25			
4	5	18.035039	2012-06-14 12:13:30	15.026711	3	1 0
5	6	23.042624	2012-06-14 12:13:35	20.034296	3	1 0
6	7	28.050181	2012-06-14 12:13:40	25.041853	3	1 0
7	8	33.057714	2012-06-14 12:13:45	30.049386	3	1 0
8	9	38.065421	2012-06-14 12:13:50	35.057093	3	1 0
9	10	43.072973	2012-06-14 12:13:55	40.064645	3	1 0

In [50]:

low_curr_ocv_20.describe()

Out[50]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	31018.000000	31018.000000	31018	31018.000000	31018.000000
mean	15509.500000	82530.938666	2012-06-15 11:08:43.815655168	37436.752180	5.955671
min	1.000000	3.007810	2012-06-14 12:13:15	0.000002	1.000000
25%	7755.250000	42125.027738	2012-06-14 23:55:17.249999872	17983.441759	5.000000
50%	15509.500000	80954.993049	2012-06-15 10:42:27.500000	37399.049136	5.000000
75%	23263.750000	123082.373990	2012-06-15 22:24:35.750000128	56814.656519	7.000000
max	31018.000000	165211.592285	2012-06-16 10:06:45	76708.858391	8.000000
std	8954.269661	46356.820098	NaN	22362.338814	1.053492

In [51]:

low_curr_ocv_20.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31018 entries, 0 to 31017
Data columns (total 19 columns):
Column Non-Null Count Dtype
--- -

5/19/25, 1:30 AM

Assignments/A123 battery/low ocv curr -10.ipynb at master · Jai-Kumar786/Assignments

```
0 Data_Point 31018 non-null int64
1 Test_Time(s) 31018 non-null float64
2 Date_Time 31018 non-null datetime64[ns]
3 Step_Time(s) 31018 non-null float64
4 Step_Index 31018 non-null int64
5 Cycle_Index 31018 non-null int64
6 Current(A) 31018 non-null float64
7 Voltage(V) 31018 non-null float64
8 Charge_Capacity(Ah) 31018 non-null float64
9 Discharge_Capacity(Ah) 31018 non-null float64
10 Charge_Energy(Wh) 31018 non-null float64
11 Discharge_Energy(Wh) 31018 non-null float64
12 dV/dt(V/s) 31018 non-null float64
13 Internal_Resistance(Ohm) 31018 non-null int64
14 Is_FC_Data 31018 non-null int64
15 AC_Impedance(Ohm) 31018 non-null int64
16 ACI_Phase_Angle(Deg) 31018 non-null int64
17 Temperature (C)_1 31018 non-null float64
18 Temperature (C)_2 31018 non-null float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.5 MB
```

LOW CURRENT OCV AT 25 degree celcius

In [52]:

```
low_curr_ocv_25=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\
```

In [53]:

```
low_curr_ocv_25.head(10)
```

Out[53]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.003881	2012-09-05 09:50:31	3.003881	1	1	0.
1	2	8.004174	2012-09-05 09:50:36	5.000008	2	1	1.
2	3	13.004248	2012-09-05 09:50:41	10.000082	2	1	1.
3	4	18.004260	2012-09-05 09:50:46	15.000093	2	1	1.
4	5	23.005579	2012-09-05 09:50:51	20.001413	2	1	1.
5	6	28.005657	2012-09-05 09:50:56	25.001492	2	1	1.
6	7	33.011929	2012-09-05 09:51:01	30.007763	2	1	1.

7	8	38.011951	2012-09-05	35.007785	2	1	1.
			09:51:06				
8	9	43.011997	2012-09-05	40.007831	2	1	1.
			09:51:11				
9	10	48.012036	2012-09-05	45.007870	2	1	1.
			09:51:16				



In [54]:

low_curr_ocv_25.describe()

Out[54]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	32307.00000	32307.000000	32307	32307.000000	32307.000000
mean	16154.00000	85506.863991	2012-09-06 09:35:35.959884800	36302.546619	5.837713
min	1.00000	3.003881	2012-09-05 09:50:31	3.003881	1.000000
25%	8077.50000	43696.841249	2012-09-05 21:58:45.500000	15874.938811	5.000000
50%	16154.00000	84091.821468	2012-09-06 09:12:01	36076.586865	5.000000
75%	24230.50000	127800.461601	2012-09-06 21:20:30.500000	56272.784453	7.000000
max	32307.00000	171495.535618	2012-09-07 09:28:46	76592.209924	8.000000
std	9326.37191	48357.812231	NaN	22968.249246	1.217815



In [55]:

low_curr_ocv_25.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32307 entries, 0 to 32306
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            32307 non-null  int64
1   Test_Time(s)                          32307 non-null  float64
2   Date_Time                             32307 non-null  datetime64[ns]
3   Step_Time(s)                          32307 non-null  float64
4   Step_Index                            32307 non-null  int64
5   Cycle_Index                           32307 non-null  int64
6   Current(A)                            32307 non-null  float64
7   Voltage(V)                            32307 non-null  float64
8   Charge_Capacity(Ah)                   32307 non-null  float64
9   Discharge_Capacity(Ah)                32307 non-null  float64
10  Charge_Energy(Wh)                     32307 non-null  float64
11  Discharge_Energy(Wh)                   32307 non-null  float64
```

```
12 dV/dt(V/s) 32307 non-null float64
13 Internal_Resistance(Ohm) 32307 non-null int64
14 Is_FC_Data 32307 non-null int64
15 AC_Impedance(Ohm) 32307 non-null int64
16 ACI_Phase_Angle(Deg) 32307 non-null int64
17 Temperature (C)_1 32307 non-null float64
dtypes: datetime64[ns](1), float64(10), int64(7)
memory usage: 4.4 MB
```

LOW CURRENT OCV AT 30 degree celcius

```
In [56]: low_curr_ocv_30=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\
```

```
In [57]: low_curr_ocv_30.head(10)
```

```
Out[57]:
```

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.010808	2012-06-25 10:58:13	3.010808	1	1	0.
1	2	3.041952	2012-06-25 10:58:13	0.030896	2	1	1.
2	3	8.057809	2012-06-25 10:58:18	5.015575	3	1	0.
3	4	13.065362	2012-06-25 10:58:23	10.023128	3	1	0.
4	5	18.072954	2012-06-25 10:58:28	15.030720	3	1	0.
5	6	23.080509	2012-06-25 10:58:33	20.038275	3	1	0.
6	7	28.088092	2012-06-25 10:58:38	25.045859	3	1	0.
7	8	33.095648	2012-06-25 10:58:43	30.053414	3	1	0.
8	9	38.103228	2012-06-25 10:58:48	35.060994	3	1	0.
9	10	43.110795	2012-06-25 10:58:53	40.068561	3	1	0.

```
In [58]: low_curr_ocv_30.describe()
```

Out[58]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	31150.000000	31150.000000	31150	31150.000000	31150.000000
mean	15575.500000	82879.119805	2012-06-26 09:59:30.199871488	37868.880320	5.967159
min	1.000000	3.010808	2012-06-25 10:58:13	0.030896	1.000000
25%	7788.250000	42289.364036	2012-06-25 22:43:00.24999872	18343.988166	5.000000
50%	15575.500000	81284.579150	2012-06-26 09:32:55.500000	37842.221660	5.000000
75%	23362.750000	123576.509714	2012-06-26 21:17:47.750000128	57340.455169	7.000000
max	31150.000000	165868.338984	2012-06-27 09:02:40	77269.004464	8.000000
std	8992.374779	46527.087207	NaN	22474.828502	1.040072

```
In [59]: low_curr_ocv_30.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31150 entries, 0 to 31149
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            31150 non-null  int64
1   Test_Time(s)                          31150 non-null  float64
2   Date_Time                             31150 non-null  datetime64[ns]
3   Step_Time(s)                          31150 non-null  float64
4   Step_Index                            31150 non-null  int64
5   Cycle_Index                           31150 non-null  int64
6   Current(A)                            31150 non-null  float64
7   Voltage(V)                            31150 non-null  float64
8   Charge_Capacity(Ah)                   31150 non-null  float64
9   Discharge_Capacity(Ah)                31150 non-null  float64
10  Charge_Energy(Wh)                     31150 non-null  float64
11  Discharge_Energy(Wh)                   31150 non-null  float64
12  dV/dt(V/s)                            31150 non-null  float64
13  Internal_Resistance(Ohm)                31150 non-null  int64
14  Is_FC_Data                             31150 non-null  int64
15  AC_Impedance(Ohm)                      31150 non-null  int64
16  ACI_Phase_Angle(Deg)                   31150 non-null  int64
17  Temperature (C)_1                       31150 non-null  float64
18  Temperature (C)_2                       31150 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.5 MB
```

LOW CURRENT OCV AT 40 degree

ceicius

In [60]:

low_curr_ocv_40=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\

In [61]:

low_curr_ocv_40.head(10)

Out[61]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index	Cycle_Index	Cur
0	1	3.010283	2012-06-27 11:43:25	3.010283	1	1	0.
1	2	3.026131	2012-06-27 11:43:25	0.015609	2	1	1.
2	3	8.033910	2012-06-27 11:43:30	5.007549	3	1	0.
3	4	13.041500	2012-06-27 11:43:35	10.015140	3	1	0.
4	5	18.049071	2012-06-27 11:43:40	15.022711	3	1	0.
5	6	23.056643	2012-06-27 11:43:45	20.030282	3	1	0.
6	7	28.064215	2012-06-27 11:43:50	25.037854	3	1	0.
7	8	33.071785	2012-06-27 11:43:55	30.045425	3	1	0.
8	9	38.079286	2012-06-27 11:44:00	35.052926	3	1	0.
9	10	43.086932	2012-06-27 11:44:05	40.060571	3	1	0.

In [62]:

low_curr_ocv_40.describe()

Out[62]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	31258.000000	31258.000000	31258	31258.000000	31258.000000
mean	15629.500000	83182.146533	2012-06-28 10:49:45.445390080	38403.674272	5.985124

min	1.000000	3.010283	2012-06-27 11:43:25	0.015609	1.000000
25%	7815.250000	42427.009169	2012-06-27 23:30:30.249999872	18823.463111	5.000000
50%	15629.500000	81557.428742	2012-06-28 10:22:40.500000	38388.046949	5.000000
75%	23443.750000	123985.717499	2012-06-28 22:09:49.750000128	57952.630847	7.000000
max	31258.000000	166414.916638	2012-06-29 09:56:59	77785.938692	8.000000
std	9023.551694	46653.945691	NaN	22568.608281	1.019973



In [63]:

```
low_curr_ocv_40.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31258 entries, 0 to 31257
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            31258 non-null  int64
1   Test_Time(s)                         31258 non-null  float64
2   Date_Time                            31258 non-null  datetime64[ns]
3   Step_Time(s)                         31258 non-null  float64
4   Step_Index                           31258 non-null  int64
5   Cycle_Index                           31258 non-null  int64
6   Current(A)                           31258 non-null  float64
7   Voltage(V)                           31258 non-null  float64
8   Charge_Capacity(Ah)                  31258 non-null  float64
9   Discharge_Capacity(Ah)               31258 non-null  float64
10  Charge_Energy(Wh)                    31258 non-null  float64
11  Discharge_Energy(Wh)                 31258 non-null  float64
12  dV/dt(V/s)                           31258 non-null  float64
13  Internal_Resistance(Ohm)              31258 non-null  int64
14  Is_FC_Data                           31258 non-null  int64
15  AC_Impedance(Ohm)                    31258 non-null  int64
16  ACI_Phase_Angle(Deg)                 31258 non-null  int64
17  Temperature (C)_1                     31258 non-null  float64
18  Temperature (C)_2                     31258 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.5 MB
```

LOW CURRENT OCV AT 50 degree celcius

In [64]:

```
low_curr_ocv_50=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\
```



In [65]:

```
low_curr_ocv_50.head(10)
```

Out[65]:

```
Data_Point  Test_Time(s)  Date_Time  Step_Time(s)  Step_Index  Cycle_Index  Cur
```

0	1	3.008714	2012-07-02 10:52:00	3.008714	1	1	0
1	2	3.020297	2012-07-02 10:52:00	0.000002	2	1	1
2	3	8.037763	2012-07-02 10:52:05	5.007566	3	1	0
3	4	13.045325	2012-07-02 10:52:10	10.015128	3	1	0
4	5	18.052910	2012-07-02 10:52:15	15.022712	3	1	0
5	6	23.060484	2012-07-02 10:52:20	20.030287	3	1	0
6	7	28.068046	2012-07-02 10:52:25	25.037848	3	1	0
7	8	33.075622	2012-07-02 10:52:30	30.045425	3	1	0
8	9	38.083297	2012-07-02 10:52:35	35.053100	3	1	0
9	10	43.090758	2012-07-02 10:52:40	40.060561	3	1	0



In [66]: `low_curr_ocv_50.describe()`

Out[66]:

	Data_Point	Test_Time(s)	Date_Time	Step_Time(s)	Step_Index
count	31475.000000	31475.000000	31475	31475.000000	31475.000000
mean	15738.000000	83704.689997	2012-07-03 10:07:02.332041216	38368.988081	5.975473
min	1.000000	3.008714	2012-07-02 10:52:00	0.000002	1.000000
25%	7869.500000	42695.210585	2012-07-02 22:43:32.500000	18645.694362	5.000000
50%	15738.000000	82097.290906	2012-07-03 09:40:15	38347.986437	5.000000
75%	23606.500000	124794.372180	2012-07-03 21:31:52.500000	58047.774699	7.000000

max 31475.000000 167494.127364 2012-07-04 09:23:33 77843.159249 8.000000

std 9086.194198 46988.606063 NaN 22713.080086 1.035112

In [67]:

```
low_curr_ocv_50.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31475 entries, 0 to 31474
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Data_Point                            31475 non-null  int64
1   Test_Time(s)                          31475 non-null  float64
2   Date_Time                             31475 non-null  datetime64[ns]
3   Step_Time(s)                          31475 non-null  float64
4   Step_Index                            31475 non-null  int64
5   Cycle_Index                           31475 non-null  int64
6   Current(A)                            31475 non-null  float64
7   Voltage(V)                            31475 non-null  float64
8   Charge_Capacity(Ah)                   31475 non-null  float64
9   Discharge_Capacity(Ah)                 31475 non-null  float64
10  Charge_Energy(Wh)                      31475 non-null  float64
11  Discharge_Energy(Wh)                   31475 non-null  float64
12  dV/dt(V/s)                            31475 non-null  float64
13  Internal_Resistance(Ohm)                31475 non-null  int64
14  Is_FC_Data                             31475 non-null  int64
15  AC_Impedance(Ohm)                      31475 non-null  int64
16  ACI_Phase_Angle(Deg)                   31475 non-null  int64
17  Temperature (C)_1                      31475 non-null  float64
18  Temperature (C)_2                      31475 non-null  float64
dtypes: datetime64[ns](1), float64(11), int64(7)
memory usage: 4.6 MB
```

DYNAMIC TEST PROFILES

Dynamic test profile at -10 degree celcius

In [68]:

```
dst_minus_10=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A12
```

In [69]:

```
dst_minus_10.head(10)
```

Out[69]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.003816	3.003816	1	0.000000	2.803019	-8.300812
1	8.004204	5.000077	2	-1.099373	2.109292	-8.145514
2	12.006524	9.002396	2	-1.099373	1.999416	-8.145514
3	17.013409	5.003358	3	0.000000	2.671906	-8.145514
4	22.015560	10.005508	3	0.000000	2.689757	-8.223158

5	27.017961	15.007909	3	0.000000	2.697759	-8.191391
6	32.018054	20.008003	3	0.000000	2.702376	-8.223158
7	37.026317	25.016265	3	0.000000	2.706993	-8.113752
8	42.030830	30.020778	3	0.000000	2.709763	-8.067879
9	47.039970	35.029918	3	0.000000	2.713148	-8.223158

In [70]:

dst_minus_10.describe()

Out[70]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	19784.000000	1.978400e+04	19784.000000	19784.000000	19784.000000	1978
mean	20198.292967	8.537425e+02	15.393095	-0.322788	3.165822	-
std	9740.553386	1.314745e+03	6.824850	0.887062	0.304046	-
min	3.003816	3.849596e-07	1.000000	-3.849595	1.999416	-
25%	11169.170425	1.615422e+02	8.000000	-0.758130	2.985145	-
50%	20727.640514	3.380252e+02	16.000000	-0.008637	3.164809	-
75%	30207.416255	8.841617e+02	24.000000	0.134114	3.347782	-
max	35460.091077	7.315288e+03	27.000000	2.061261	3.814677	-

In [71]:

dst_minus_10.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19784 entries, 0 to 19783
Data columns (total 6 columns):
Column Non-Null Count Dtype
--- ---
0 Test_Time(s) 19784 non-null float64
1 Step_Time(s) 19784 non-null float64
2 Step_Index 19784 non-null int64
3 Current(A) 19784 non-null float64
4 Voltage(V) 19784 non-null float64
5 Temperature (C)_1 19784 non-null float64
dtypes: float64(5), int64(1)
memory usage: 927.5 KB

Dynamic test profile at 0 degree celcius

In [72]:

dst_0=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batte

In [73]:

dst_0.head(10)

Out[73]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
--	--------------	--------------	------------	------------	------------	-------------------

0	3.057882	3.005927	1	0.000000	2.846107	1.256174
1	8.065274	5.007092	2	-1.099185	2.468466	1.256174
2	13.072544	10.014362	2	-1.099373	2.403833	1.332009
3	18.077826	15.019644	2	-1.099373	2.333968	1.256174
4	23.078087	20.019905	2	-1.099373	2.250561	1.301033
5	28.082401	25.024220	2	-1.099560	2.134222	1.452703
6	31.764519	28.706338	2	-1.099373	1.999416	1.345893
7	36.774210	5.008228	3	0.000000	2.548488	1.332009
8	41.783489	10.017507	3	0.000000	2.566955	1.376867
9	46.792691	15.026708	3	0.000000	2.577727	1.332009

In [74]:

dst_0.describe()

Out[74]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	23248.000000	2.324800e+04	23248.000000	23248.000000	23248.000000	2324
mean	20466.229194	5.328935e+02	15.566027	-0.348093	3.165784	
std	10301.715583	6.231313e+02	6.763405	0.918652	0.259291	
min	3.057882	7.699193e-07	1.000000	-3.849219	1.961867	-
25%	10658.417985	1.537679e+02	8.000000	-0.829459	3.026156	
50%	20792.540701	3.166302e+02	16.000000	-0.191961	3.172349	
75%	30805.391681	6.670666e+02	24.000000	0.080958	3.293613	
max	36926.114253	4.328226e+03	27.000000	2.060698	3.728192	

In [75]:

dst_0.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23248 entries, 0 to 23247
Data columns (total 6 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Test_Time(s)                         23248 non-null  float64
1   Step_Time(s)                         23248 non-null  float64
2   Step_Index                           23248 non-null  int64
3   Current(A)                           23248 non-null  float64
4   Voltage(V)                           23248 non-null  float64
5   Temperature (C)_1                    23248 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

Dynamic test profile at 10 degree celcius

In [76]:

dst_10=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batt

In [77]: `dst_10.head(10)`

Out[77]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.008291	3.008291	1	0.000000	2.968602	9.646121
1	8.015758	5.007161	2	-1.099185	2.729152	9.690903
2	13.018937	10.010341	2	-1.099373	2.703299	9.598107
3	18.027239	15.018642	2	-1.099373	2.678369	9.797547
4	23.031320	20.022724	2	-1.099373	2.654363	9.766615
5	28.035736	25.027140	2	-1.099373	2.629125	9.721836
6	33.039112	30.030516	2	-1.099373	2.603272	9.797547
7	38.047386	35.038790	2	-1.099373	2.576496	9.690903
8	43.053577	40.044981	2	-1.099373	2.547873	9.918034
9	48.062643	45.054046	2	-1.099373	2.515864	9.873258

In [78]: `dst_10.describe()`

Out[78]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	23881.000000	2.388100e+04	23881.000000	23881.000000	23881.000000	2388
mean	19919.260228	4.755559e+02	15.518613	-0.356659	3.190354	1
std	10203.319502	4.779053e+02	6.705619	0.932044	0.224014	
min	3.008291	7.699193e-07	1.000000	-3.849407	1.987413	
25%	10199.540085	1.497195e+02	8.000000	-0.848430	3.077862	1
50%	20191.833861	3.095074e+02	16.000000	-0.243238	3.196048	1
75%	30156.128886	5.997450e+02	24.000000	0.072130	3.302231	1
max	36435.385542	2.681824e+03	27.000000	2.061261	3.699877	1

In [79]: `dst_10.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23881 entries, 0 to 23880
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Test_Time(s)          23881 non-null  float64
1   Step_Time(s)          23881 non-null  float64
2   Step_Index            23881 non-null  int64
3   Current(A)            23881 non-null  float64
4   Voltage(V)            23881 non-null  float64
```

```
5 Temperature (C)_1 23881 non-null float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

Dynamic test profile at 20 degree celcius

```
In [80]: dst_20=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batt
```

```
In [81]: dst_20.head(10)
```

```
Out[81]:
```

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.008332	3.008332	1	0.000000	3.108637	18.427523
1	8.014952	5.006227	2	-1.099776	2.891659	18.444551
2	13.018221	10.009497	2	-1.099776	2.877501	18.427523
3	18.022437	15.013712	2	-1.099957	2.864575	18.489090
4	23.026696	20.017971	2	-1.099776	2.851957	18.382978
5	28.035930	25.027205	2	-1.099957	2.839954	18.458305
6	33.045322	30.036597	2	-1.099776	2.827950	18.365955
7	38.051520	35.042795	2	-1.099957	2.815948	18.427523
8	43.066605	40.057881	2	-1.099776	2.803944	18.352194
9	48.077106	45.068381	2	-1.099776	2.791941	18.321405

```
In [82]: dst_20.describe()
```

```
Out[82]:
```

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	24992.000000	2.499200e+04	24992.000000	24992.000000	24992.000000	2499
mean	20478.506604	4.711195e+02	15.573944	-0.362114	3.195718	2
std	10524.199546	4.800529e+02	6.722357	0.935984	0.229664	
min	3.008332	7.699193e-07	1.000000	-3.849713	1.898483	1
25%	10446.853748	1.524163e+02	8.000000	-0.862705	3.091094	1
50%	20715.938621	3.099327e+02	16.000000	-0.279794	3.208663	1
75%	30962.624857	5.888735e+02	24.000000	0.063971	3.315767	2
max	37521.292897	2.867548e+03	27.000000	2.061054	3.732181	2

```
In [83]: dst_20.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 24992 entries, 0 to 24991
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Test_Time(s)          24992 non-null  float64
1   Step_Time(s)          24992 non-null  float64
2   Step_Index            24992 non-null  int64
3   Current(A)            24992 non-null  float64
4   Voltage(V)            24992 non-null  float64
5   Temperature (C)_1     24992 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

Dynamic test profile at 25 degree celcius

In [84]:

dst_25=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batt

In [85]:

dst_25.head(10)

Out[85]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.053504	3.000204	1	0.000000	2.766393	26.788187
1	8.061154	5.007273	2	-1.099373	2.501398	26.818743
2	13.062293	10.008411	2	-1.099185	2.435226	26.818743
3	18.071620	15.017739	2	-1.099373	2.340432	26.713451
4	23.073773	20.019892	2	-1.099185	2.115448	26.788187
5	24.301853	21.247971	2	-1.099373	1.998801	26.893473
6	29.307341	5.004141	3	0.000000	2.460464	26.893473
7	34.311587	10.008388	3	0.000000	2.474314	26.713451
8	39.319818	15.016618	3	0.000000	2.482316	26.818743
9	44.325095	20.021896	3	0.000000	2.486933	26.937641

In [86]:

dst_25.describe()

Out[86]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	24469.000000	2.446900e+04	24469.000000	24469.000000	24469.000000	2446
mean	19726.353242	4.718362e+02	15.557767	-0.363447	3.204312	2
std	10192.971787	5.018513e+02	6.710170	0.942908	0.211759	
min	3.053504	3.849596e-07	1.000000	-3.849407	1.937861	2
25%	10017.592323	1.497245e+02	8.000000	-0.867589	3.110487	2
50%	19958.708627	3.061075e+02	16.000000	-0.286440	3.214515	2
75%	29860.944817	5.805784e+02	24.000000	0.060672	3.311772	2

max 36294.795004 3.070756e+03 27.000000 2.061261 3.699877 2

In [87]:

```
dst_25.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24469 entries, 0 to 24468
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Test_Time(s)          24469 non-null  float64
1   Step_Time(s)          24469 non-null  float64
2   Step_Index            24469 non-null  int64   
3   Current(A)            24469 non-null  float64
4   Voltage(V)            24469 non-null  float64
5   Temperature (C)_1     24469 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

Dynamic test profile at 30 degree celcius

In [88]:

```
dst_30=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batt
```

In [89]:

```
dst_30.head(10)
```

Out[89]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.004871	3.004871	1	0.000000	3.075400	31.391201
1	8.011347	5.006164	2	-1.099373	2.888580	31.242373
2	13.017574	10.012391	2	-1.099373	2.876269	31.391201
3	18.023831	15.018647	2	-1.099373	2.864882	31.421654
4	23.027999	20.022816	2	-1.099373	2.853494	31.496058
5	28.032410	25.027227	2	-1.099373	2.842414	31.421654
6	33.039675	30.034492	2	-1.099373	2.831334	31.421654
7	38.041781	35.036597	2	-1.099373	2.819946	31.465605
8	43.043244	40.038061	2	-1.099373	2.808558	31.465605
9	48.043298	45.038114	2	-1.099560	2.796555	31.391201

In [90]:

```
dst_30.describe()
```

Out[90]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	24598.000000	2.459800e+04	24598.000000	24598.000000	24598.000000	2459
mean	19621.906470	4.721762e+02	15.567973	-0.364780	3.206956	3

std	10170.231343	5.126144e+02	6.716462	0.942931	0.203930	
min	3.004871	3.849596e-07	1.000000	-3.849407	1.949556	3
25%	9938.020160	1.497872e+02	8.000000	-0.871909	3.116026	3
50%	19831.893588	3.053628e+02	16.000000	-0.297146	3.217592	3
75%	29711.598095	5.757773e+02	24.000000	0.049966	3.312695	3
max	36178.975170	3.143676e+03	27.000000	2.061261	3.696491	3

In [91]:

dst_30.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24598 entries, 0 to 24597
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Test_Time(s)          24598 non-null  float64
1   Step_Time(s)          24598 non-null  float64
2   Step_Index            24598 non-null  int64
3   Current(A)            24598 non-null  float64
4   Voltage(V)            24598 non-null  float64
5   Temperature (C)_1     24598 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

Dynamic test profile at 50 degree celcius

In [92]:

dst_50=pd.read_excel(r"C:\Users\jaiku\PycharmProjects\Assignments\A123 batt

In [93]:

dst_50.head(10)

Out[93]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Temperature (C)_1
0	3.007451	3.007451	1	0.000000	2.775626	50.789497
1	8.012196	5.004451	2	-1.099373	2.564185	50.789497
2	13.012202	10.004457	2	-1.099373	2.523558	50.789497
3	18.014646	15.006900	2	-1.099373	2.473083	50.746269
4	23.021725	20.013979	2	-1.099373	2.404141	50.879433
5	28.021949	25.014203	2	-1.099185	2.257025	50.952637
6	30.994169	27.986424	2	-1.099185	1.999724	50.746269
7	35.996811	5.001098	3	0.000000	2.367208	50.836208
8	41.000069	10.004355	3	0.000000	2.385059	50.866188
9	46.000322	15.004608	3	0.000000	2.393369	50.819477

In [94]: `dst_50.describe()`

Out[94]:

	Test_Time(s)	Step_Time(s)	Step_Index	Current(A)	Voltage(V)	Tem
count	24763.000000	2.476300e+04	24763.000000	24763.000000	24763.000000	2476
mean	19197.121372	4.804346e+02	15.652263	-0.367141	3.214068	5
std	10034.610605	5.380568e+02	6.695618	0.947778	0.195042	
min	3.007451	3.849596e-07	1.000000	-3.849407	1.926166	5
25%	9617.456118	1.515973e+02	8.000000	-0.880455	3.126183	5
50%	19349.539252	3.072322e+02	16.000000	-0.311609	3.226826	5
75%	29076.097827	5.734703e+02	24.000000	0.034939	3.320697	5
max	35582.345606	3.301138e+03	27.000000	2.061261	3.709726	5

In [95]: `dst_50.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 24763 entries, 0 to 24762
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Test_Time(s)          24763 non-null  float64
1   Step_Time(s)          24763 non-null  float64
2   Step_Index            24763 non-null  int64
3   Current(A)            24763 non-null  float64
4   Voltage(V)            24763 non-null  float64
5   Temperature (C)_1     24763 non-null  float64
dtypes: float64(5), int64(1)
memory usage: 1.1 MB
```

In [96]:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib.ticker import ScalarFormatter
from matplotlib.gridspec import GridSpec

# Set style for better visualization
plt.style.use('seaborn-v0_8-whitegrid')

# Define color map for temperatures
temp_colors = {
    '-10': '#0033A0', # Deep blue
    '0': '#0066CC', # Blue
    '10': '#3399FF', # Light blue
    '20': '#66CC00', # Green
    '25': '#FFCC00', # Yellow
    '30': '#FF9900', # Orange
    '40': '#FF6600', # Dark orange
    '50': '#CC0000', # Red
}

# List of all temperature datasets
temp_datasets = {
```



```

-10': low_curr_ocv_minus_10,
'0': low_curr_ocv_0,
'10': low_curr_ocv_10,
'20': low_curr_ocv_20,
'25': low_curr_ocv_25,
'30': low_curr_ocv_30,
'40': low_curr_ocv_40,
'50': low_curr_ocv_50
}

```

1. Charge Capacity vs Cycle Function

This plot shows how the battery's charge capacity changes over cycles at different temperatures:

In [133...

```

def plot_charge_capacity_vs_cycle(datasets, temp_colors):
    """
    Plot charge capacity vs cycle for different temperatures.

    Parameters:
    -----
    datasets : dict
        Dictionary with temperature as keys and dataframes as values
    temp_colors : dict
        Dictionary with temperature as keys and color codes as values
    """
    plt.figure(figsize=(12, 8))

    for temp, df in datasets.items():
        # Group by cycle and get max charge capacity for each cycle
        cycle_data = df.groupby('Cycle_Index')['Discharge_Capacity(Ah)'].max()

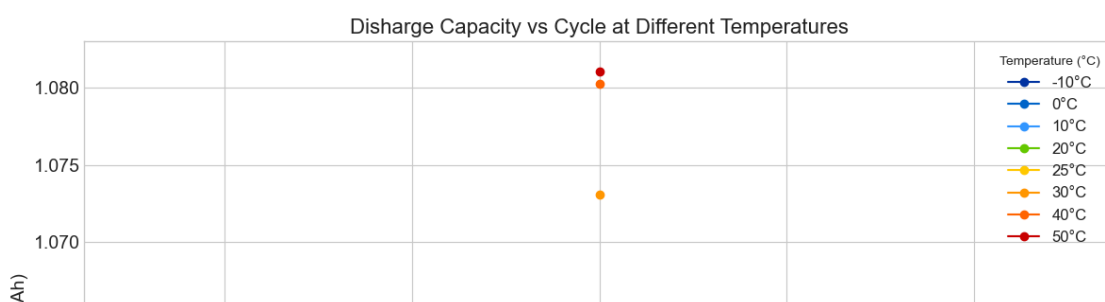
        plt.plot(cycle_data['Cycle_Index'], cycle_data['Discharge_Capacity(Ah)'],
                 marker='o', linestyle='-', label=f'{temp}°C', color=temp_colors[temp])

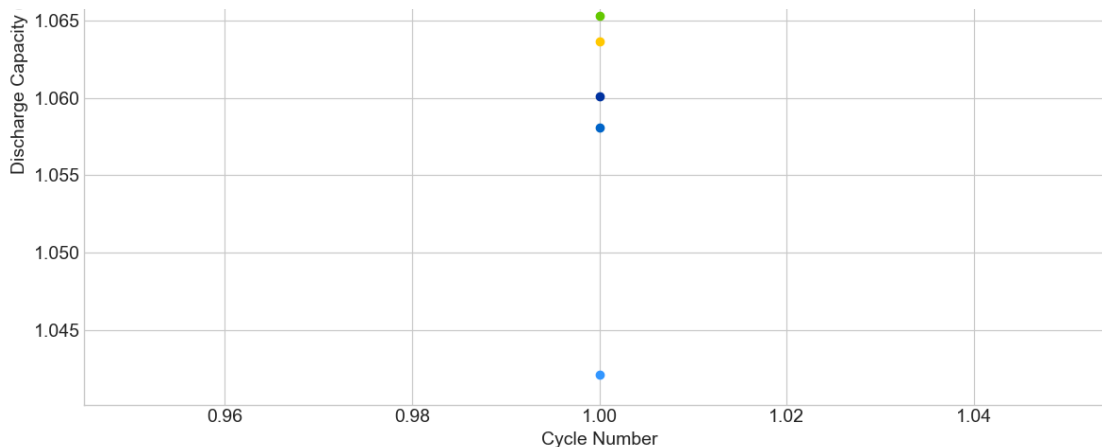
    plt.title('Discharge Capacity vs Cycle at Different Temperatures', fontsize=14)
    plt.xlabel('Cycle Number', fontsize=14)
    plt.ylabel('Discharge Capacity (Ah)', fontsize=14)
    plt.grid(True)
    plt.legend(fontsize=12, title='Temperature (°C)')
    plt.tight_layout()
    plt.rcParams['xtick.labelsize'] = 14 # X-axis tick labels
    plt.rcParams['ytick.labelsize'] = 14 # Y-axis tick labels
    # Save figure
    plt.savefig('discharge_capacity_vs_cycle.png', dpi=300, bbox_inches='tight')
    plt.show()

```

In [122...

```
plot_charge_capacity_vs_cycle(temp_datasets, temp_colors)
```





Understanding Battery Performance at Different Temperatures

What This Plot Shows

This graph illustrates how temperature affects a lithium-ion battery's discharge capacity (specifically an A123 battery). Though we're only seeing data from a single battery cycle, the plot reveals an important relationship between temperature and battery performance.

Simple Explanation

What is Discharge Capacity?

Discharge capacity (measured in Ampere-hours or Ah) is essentially how much energy a battery can deliver before running out. Think of it like the size of your gas tank - a higher capacity means your device runs longer before needing a recharge.

Key Observations

Looking at the data points on the graph:

- **Warm batteries perform better:** The batteries at 40°C and 50°C (red and orange dots) have the highest discharge capacity (around 1.080 Ah)
- **Cold batteries perform worse:** The battery at -10°C (darkest blue dot) has the lowest capacity (around 1.042 Ah)
- **Clear temperature pattern:** As temperature increases from cold to warm, the battery capacity consistently increases

Real-World Impact

This 3-4% difference in capacity might seem small, but imagine:

- Your phone dying 3-4% faster in cold weather
- Your electric car's range dropping noticeably in winter
- Your laptop battery not lasting as long when used outdoors in cold

temperatures

Engineering Insights

Why Temperature Affects Batteries

Temperature impacts how quickly chemical reactions can occur inside the battery:

1. **Cold temperatures** slow down the movement of ions (charged particles) through the battery's electrolyte, creating higher internal resistance and reducing available capacity
2. **Warm temperatures** increase ion mobility and reduce internal resistance, allowing more of the battery's stored energy to be accessible

Temperature Management Considerations

- **Optimal operating range:** This data suggests the A123 battery performs best at higher temperatures (30-50°C), but...
- **Long-term effects:** While not shown in this single-cycle data, sustained high temperatures accelerate battery degradation and shorten overall lifespan
- **Design implications:** Engineers must balance:
 - Thermal management systems that keep batteries warm in cold weather
 - Cooling systems that prevent overheating in hot conditions
 - Insulation that maintains consistent temperature

Application Insights

- **Electric vehicles:** Cold weather range reduction is a real concern, which is why many EVs have battery heating systems
- **Consumer electronics:** This explains why phones shut down faster in very cold weather, even when showing battery remaining
- **Performance/longevity trade-off:** While a 50°C battery delivers more energy initially, operating at such high temperatures would significantly reduce its lifetime

Key Takeaway

Battery performance is temperature-dependent, with warmer temperatures generally providing better immediate performance but potentially faster degradation over time. Understanding this relationship helps engineers design better battery systems and helps users understand why their battery-powered devices behave differently in various weather conditions.

2. dV/dt vs Capacity Function

This plot helps identify phase transitions during charging:

In [117...

```
def plot_dvdt_vs_capacity(datasets, temp_colors):
    """
    Plot dV/dt vs capacity for different temperatures.

    Parameters:
    -----
    datasets : dict
        Dictionary with temperature as keys and dataframes as values
    temp_colors : dict
        Dictionary with temperature as keys and color codes as values
    """
    fig = plt.figure(figsize=(16, 20))
    gs = GridSpec(2, 1, height_ratios=[1, 1])

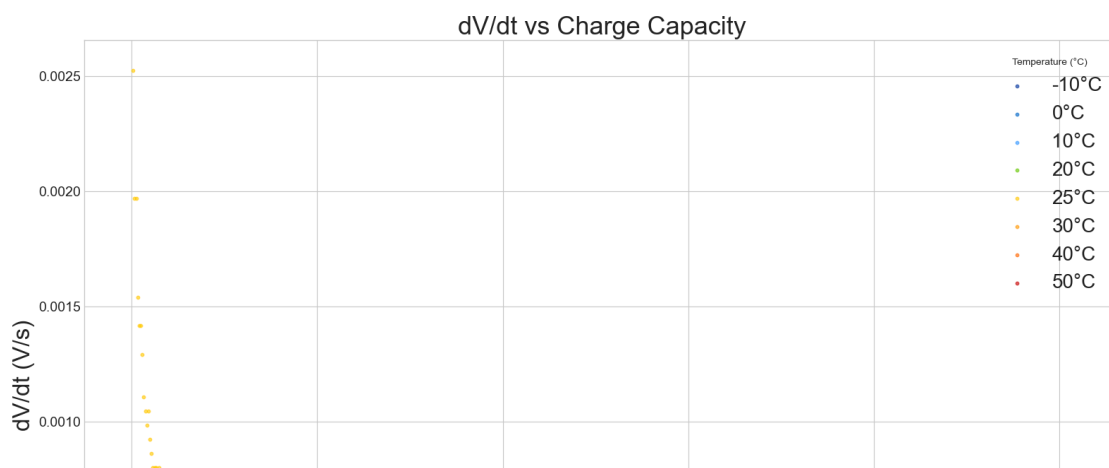
    # Plot for charging
    ax1 = fig.add_subplot(gs[0])
    for temp, df in datasets.items():
        # Filter for charging data (Current > 0)
        charge_data = df[df['Current(A)'] > 0.1].copy()
        if not charge_data.empty:
            ax1.scatter(charge_data['Charge_Capacity(Ah)'], charge_data['dV/dt (V/s)'],
                        s=10, alpha=0.6, label=f'{temp}°C', color=temp_colors[temp])

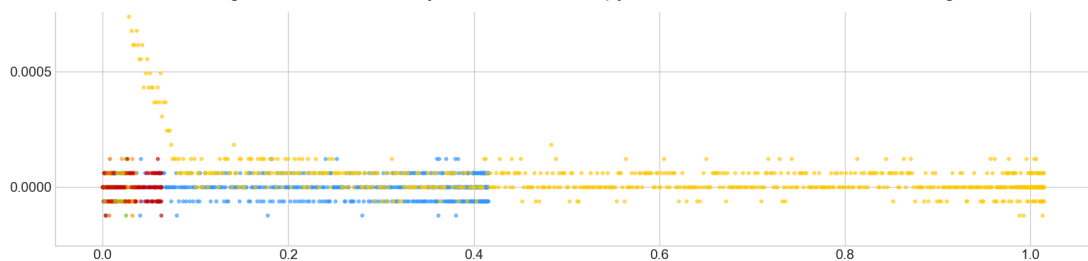
    ax1.set_title('dV/dt vs Charge Capacity', fontsize=26)
    ax1.set_ylabel('dV/dt (V/s)', fontsize=24)
    ax1.grid(True)
    ax1.legend(fontsize=20, title='Temperature (°C)')

    plt.tight_layout()
    plt.rcParams['xtick.labelsize'] = 14 # X-axis tick labels
    plt.rcParams['ytick.labelsize'] = 14 # Y-axis tick labels
    # Save figure
    plt.savefig('dvdt_vs_capacity.png', dpi=300, bbox_inches='tight')
    plt.show()
```

In [118...

```
plot_dvdt_vs_capacity(temp_datasets, temp_colors)
```





In [136...

```
fig, axes = plt.subplots(2, 4, figsize=(20, 10), sharex=True, sharey=True)
axes = axes.flatten()

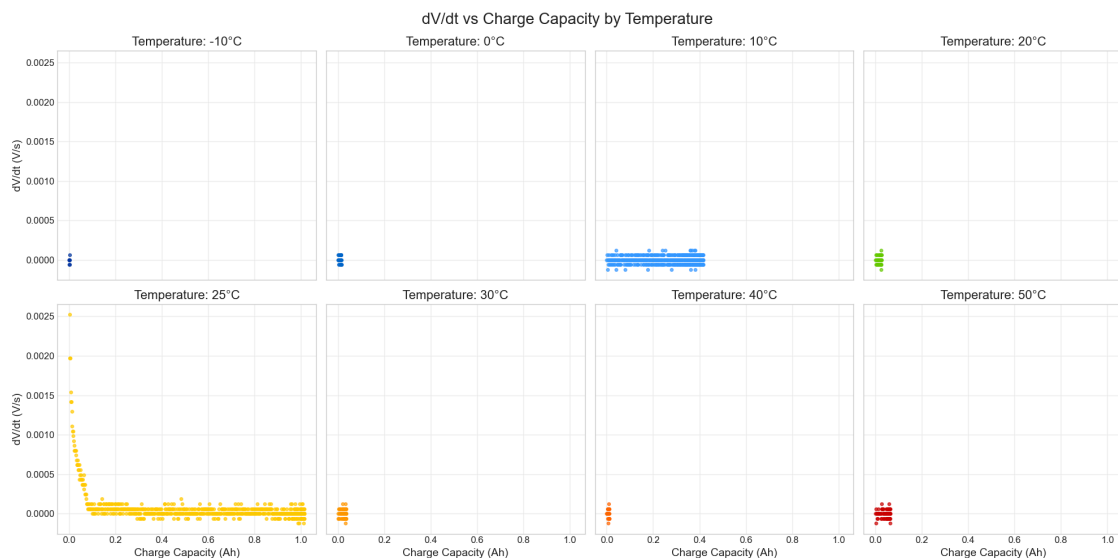
for i, (temp, df) in enumerate(sorted(temp_datasets.items())):
    ax = axes[i]
    # Filter data (remove extreme outliers)
    charge_data = df[(df['Current(A)'] > 0.1) & (abs(df['dV/dt(V/s)']) < 0.

    # Plot each temperature in its own subplot with larger points
    ax.scatter(charge_data['Charge Capacity(Ah)'], charge_data['dV/dt(V/s)'],
               color=temp_colors[temp], alpha=0.7, s=15)

    ax.set_title(f'Temperature: {temp}°C')
    ax.grid(True, alpha=0.3)

    # Only add axis labels on the outer plots
    if i >= 4: # Bottom row
        ax.set_xlabel('Charge Capacity (Ah)')
    if i % 4 == 0: # Left column
        ax.set_ylabel('dV/dt (V/s)')

plt.suptitle('dV/dt vs Charge Capacity by Temperature', fontsize=20)
plt.tight_layout()
```



Understanding Battery Charging Behavior at Different Temperatures

What We're Looking At

This set of graphs shows how quickly a battery's voltage changes (dV/dt) during

charging at eight different temperatures, from very cold (-10°C) to quite hot (50°C). Each small graph represents one temperature condition, and all eight together tell an interesting story about how batteries behave.

The Basic Idea in Simple Terms

When you charge a battery, its voltage doesn't increase at a steady rate throughout the entire process. Instead, the voltage sometimes changes quickly and sometimes changes slowly, depending on what's happening inside. These graphs track that rate of change (dV/dt) as the battery fills up (charge capacity).

Key Observations Anyone Can Understand

The Dramatic Difference at 25°C (Room Temperature)

The most striking feature is in the bottom-left graph (25°C). This shows:

- A tall column of dots at the very beginning of charging
- These dots show that when a room-temperature battery first starts charging, its voltage rises very rapidly
- As charging continues (moving right on the graph), this rapid rise quickly settles down to near-zero

Cold vs. Warm Temperatures

- At cold temperatures (-10°C , 0°C): Very few data points and minimal voltage changes
- At moderate temperatures (10°C , 20°C): More activity, but still relatively calm
- At high temperatures (30°C , 40°C , 50°C): Activity concentrated at the beginning of charging, but not nearly as dramatic as at 25°C

The "Sweet Spot" of Room Temperature

This data strongly suggests that 25°C (about 77°F) – typical room temperature – creates ideal conditions for battery charging. The chemical reactions inside the battery appear most responsive at this temperature.

Real-World Connections

This explains several everyday experiences with batteries:

- Why your devices might charge better when they're at room temperature
- Why very cold phones charge slowly and seem to have less capacity
- Why manuals often recommend charging devices at room temperature for best results

Think of it like trying to pour honey: when it's cold, it flows very slowly (low dV/dt); when it's at room temperature, it flows nicely; when it's too hot, it might flow too

quickly but less controllably.

Engineering Insights

For those interested in the technical aspects:

1. **Phase transitions:** The high dV/dt values at the beginning of charging at 25°C likely represent a phase transition in the battery materials, where lithium ions begin moving into the graphite anode.
2. **Temperature-dependent kinetics:** Chemical reactions in batteries are highly temperature-dependent. The data suggests that 25°C provides optimal energy for the charging reactions, allowing them to proceed effectively without excessive resistance (cold temperatures) or side reactions (hot temperatures).
3. **Charging algorithm implications:** Battery management systems track these voltage changes to determine optimal charging rates. This data explains why charging algorithms might apply different profiles at different temperatures.
4. **Design considerations:** Engineers designing battery systems must account for these temperature effects, perhaps by including heating elements for cold-weather operation or cooling systems for hot environments.
5. **Peak dV/dt as diagnostic tool:** The height and shape of the initial dV/dt peak could potentially serve as a diagnostic tool for battery health assessment.

Why This Matters

This visualization helps explain why:

- Electric vehicles have battery thermal management systems
- Your phone might warn you when it's too hot or cold to charge properly
- Battery performance varies with the seasons
- Manufacturers recommend specific charging conditions

The data reveals that batteries aren't simple storage containers but complex electrochemical systems whose behavior changes dramatically with temperature – with room temperature appearing to be the "Goldilocks zone" where charging behavior is most dynamic and efficient.

3 Capacity vs Voltage Function (charging capacity)

This creates voltage profiles for each temperature, showing how voltage varies with capacity during charge:

In [119...

```
def plot_capacity_vs_voltage(datasets, temp_colors):  
    """
```

Plot capacity vs voltage for both charging at different temperatures.

Parameters:

datasets : dict

Dictionary with temperature as keys and dataframes as values

temp_colors : dict

Dictionary with temperature as keys and color codes as values

"""

```
fig, ax1 = plt.subplots(1, 1, figsize=(18, 10))
```

```
# Plot for charging
```

```
for temp, df in datasets.items():
```

```
# Filter for charging data and select the first cycle for clarity
```

```
charge_data = df[(df['Current(A)'] > 0.1) & (df['Cycle_Index'] == 1)]
```

```
if not charge_data.empty:
```

```
    ax1.plot(charge_data['Charge_Capacity(Ah)'], charge_data['Voltage(V)'],
            label=f'{temp}°C', color=temp_colors[temp], linewidth=2)
```

```
ax1.set_title('Charge Capacity vs Voltage', fontsize=16)
```

```
ax1.set_xlabel('Charge Capacity (Ah)', fontsize=14)
```

```
ax1.set_ylabel('Voltage (V)', fontsize=14)
```

```
ax1.grid(True)
```

```
ax1.legend(fontsize=12, title='Temperature (°C)')
```

```
plt.tight_layout()
```

```
plt.rcParams['xtick.labelsize'] = 14 # X-axis tick labels
```

```
plt.rcParams['ytick.labelsize'] = 14 # Y-axis tick labels
```

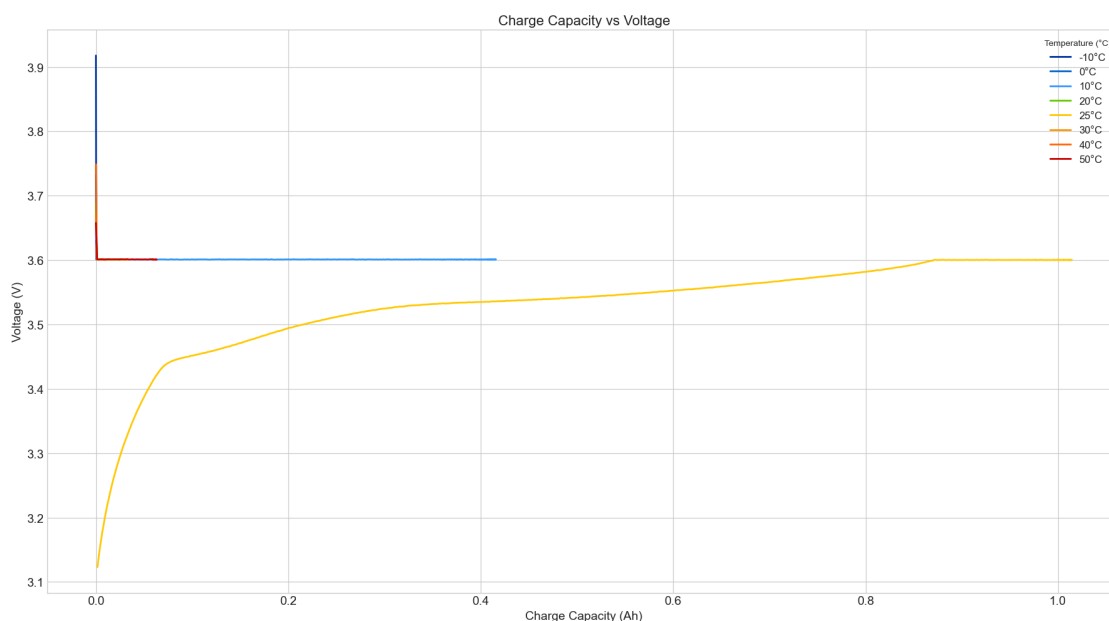
```
# Save figure
```

```
plt.savefig('capacity_vs_voltage.png', dpi=300, bbox_inches='tight')
```

```
plt.show()
```

In [120...

```
plot_capacity_vs_voltage(temp_datasets, temp_colors)
```



Understanding Battery Charging Behavior: Voltage vs. Capacity

What This Graph Shows

This graph illustrates how a battery's voltage changes as it charges at different temperatures. The horizontal axis shows the charge capacity (how "full" the battery is), while the vertical axis shows the voltage (electrical pressure) of the battery.

Key Features for Anyone to Understand

The Curious Case of Temperature Effects

The most striking feature is the dramatic difference between charging behaviors at different temperatures:

- **The yellow curve (25°C/room temperature)** shows a complete charging profile. This is what we see most clearly - a smooth curve that starts around 3.1V when empty and gradually rises to about 3.6V when full.
- **The blue line (likely -10°C or 0°C)** shows a completely different pattern - a flat line at about 3.6V. This suggests cold batteries behave differently than room-temperature ones.
- **The initial spike** at the very beginning shows a momentary voltage jump when charging first starts.

What This Means in Everyday Terms

This is similar to filling different containers with water:

- The yellow curve (room temperature battery) is like filling a vase that gets wider toward the top - it fills quickly at first (voltage rises rapidly), then more slowly as you reach the top (voltage levels off).
- The blue flat line (cold battery) is like filling a container with a completely different shape - the water level (voltage) behaves differently as you add more water (charge).

Practical Implications

1. **Temperature matters enormously** for how batteries charge. This explains why:

- Your phone might charge differently in winter vs. summer
- Electric car charging speeds vary with weather conditions
- Batteries often perform best at room temperature

2. **Voltage doesn't increase evenly** as a battery charges. This is why:

- Devices sometimes show battery percentage increasing at different rates
- Fast charging typically works best when batteries are partially empty
- Charging slows down as batteries approach full capacity

Engineering Insights

For those interested in the technical aspects:

1. **Electrochemical staging:** The gradual slope of the 25°C curve shows lithium-ion intercalation stages in the graphite anode. As the battery charges, lithium ions fill available sites in the graphite in distinct phases, creating the characteristic voltage profile.
2. **Temperature-dependent kinetics:** Cold temperatures significantly increase internal resistance and slow down ion diffusion, explaining the flat profile at low temperatures. This resistance causes energy loss as heat and reduces charging efficiency.
3. **State of charge estimation:** Battery management systems use these voltage curves as references for estimating how full a battery is. The distinctive shape at room temperature provides clear markers for different charge states.
4. **Charging protocol design:** Engineers must account for these temperature effects when designing charging algorithms. The optimal charging protocol for 25°C would be inappropriate and potentially damaging at -10°C or 50°C.
5. **Cell design implications:** Battery chemistry and physical construction significantly influence the shape of these curves. Cell designers analyze these profiles when developing new battery formulations.

Why This Matters

This graph helps explain why:

- Manufacturers specify operating temperature ranges for batteries
- Electric vehicles have sophisticated battery thermal management systems
- Quick-charging technologies can only work within specific temperature windows
- Your devices might warn you when they're too hot or cold to charge properly

The data confirms that lithium-ion batteries operate most predictably and efficiently at room temperature, with significant behavioral changes at temperature extremes. This fundamental characteristic affects everything from smartphone charging to electric grid storage systems.

Understanding the Overlapping Lines in the Battery Charging Graph

Yes, there is significant overlap between some of the temperature lines in this graph, which is an important observation! This overlap reveals crucial insights about battery charging behavior.

Where the Overlapping Occurs

Looking closely at the graph:

1. **Cold temperature lines overlap** - The blue lines (representing -10°C , 0°C , and 10°C) appear to run almost on top of each other at approximately 3.6V across much of the charging range. This suggests very similar voltage behavior during charging at these cold temperatures.
2. **Initial spike overlap** - At the very beginning of charging (near 0.0 Ah), several temperature lines start at similar points before diverging.
3. **Some warmer temperature lines** may also overlap in certain regions, though the yellow line (25°C) is most distinct.

What This Overlap Tells Us

This overlapping pattern reveals something fascinating about batteries:

1. **Temperature "bands" of similar behavior** - Batteries appear to behave in similar ways within certain temperature ranges. Cold temperatures (-10°C to 10°C) produce one type of behavior, while room temperature produces another.
2. **Threshold effects** - Rather than gradually changing behavior with each degree of temperature change, batteries seem to have threshold temperatures where their behavior changes more dramatically.
3. **Room temperature uniqueness** - The most distinct line is the yellow line (25°C /room temperature), showing this temperature creates unique charging behavior compared to both colder and hotter temperatures.

This is like how water behaves similarly as a liquid between 1°C and 99°C , but dramatically changes behavior below 0°C (ice) or above 100°C (steam). Battery charging behavior appears to have similar "state changes" at certain temperature thresholds.

Engineering Significance

The overlapping lines have important implications:

1. **Battery management simplification** - Engineers can develop charging algorithms for temperature "bands" rather than needing unique profiles for every possible temperature.
2. **Critical temperature thresholds** - The data suggests key temperature boundaries where battery behavior changes significantly, which is crucial for thermal management system design.
3. **Optimal charging window** - The distinct profile at 25°C reinforces that room temperature represents an ideal charging condition, with behavior becoming

more problematic (and more similar) at temperature extremes.

4. **Reduced data resolution at extremes** - When designing battery systems, engineers might need fewer sensors or calculations to manage cold-temperature charging but more precise controls around room temperature.

The overlapping lines aren't an error in the data or visualization—they're revealing important physical properties about how lithium-ion batteries respond to temperature during charging.

Capacity vs Voltage Function (discharging capacity)

In [123...

```
def plot_dis_capacity_vs_voltage(datasets, temp_colors):
    """
    Plot capacity vs voltage for discharging at different temperatures.

    Parameters:
    -----
    datasets : dict
        Dictionary with temperature as keys and dataframes as values
    temp_colors : dict
        Dictionary with temperature as keys and color codes as values
    """
    fig, ax1 = plt.subplots(1, 1, figsize=(18, 10))

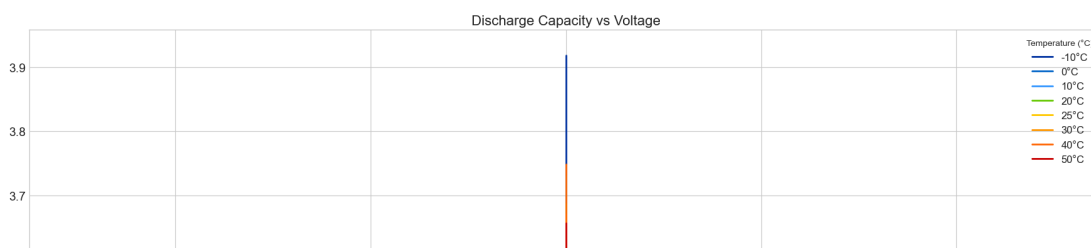
    # Plot for charging
    for temp, df in datasets.items():
        # Filter for charging data and select the first cycle for clarity
        charge_data = df[(df['Current(A)'] > 0.1) & (df['Cycle_Index'] == 1)]
        if not charge_data.empty:
            ax1.plot(charge_data['Discharge_Capacity(Ah)'], charge_data['Vo
                        label=f'{temp}°C', color=temp_colors[temp], linewidth=

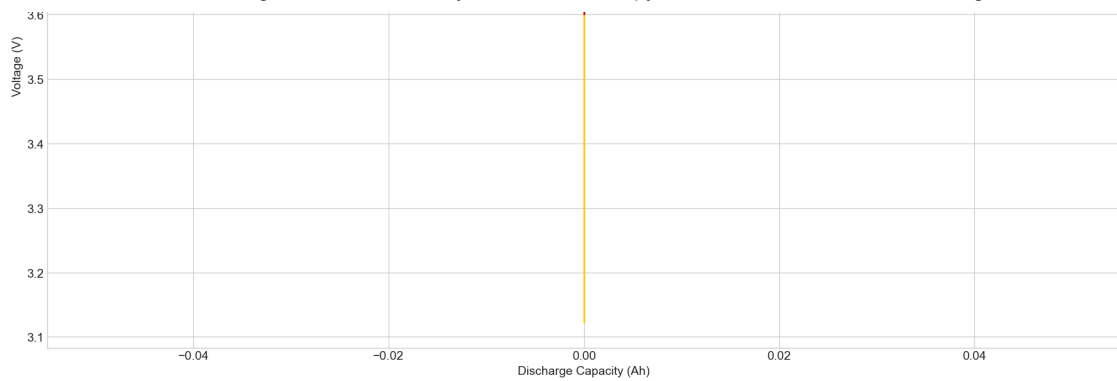
    ax1.set_title('Discharge Capacity vs Voltage', fontsize=16)
    ax1.set_xlabel('Discharge Capacity (Ah)', fontsize=14)
    ax1.set_ylabel('Voltage (V)', fontsize=14)
    ax1.grid(True)
    ax1.legend(fontsize=12, title='Temperature (°C)')

    plt.tight_layout()
    plt.rcParams['xtick.labelsize'] = 14 # X-axis tick labels
    plt.rcParams['ytick.labelsize'] = 14 # Y-axis tick labels
    # Save figure
    plt.savefig('discharge_capacity_vs_voltage.png', dpi=300, bbox_inches='
    plt.show()
```

In [124...

```
plot_dis_capacity_vs_voltage(temp_datasets, temp_colors)
```





Understanding the Battery Discharge Graph: A Vertical Line Mystery

What We're Looking At

This graph shows the relationship between "Discharge Capacity" (x-axis) and "Voltage" (y-axis) for a battery at different temperatures. However, there's something unusual happening - instead of curves extending across the graph, we see only a vertical line at zero discharge!

Explaining What We See

The Vertical Line Phenomenon

The most striking feature is that there's only a single vertical line at the 0.00 mark on the x-axis, with voltage values ranging from about 3.1V to 3.9V. The different colors represent different temperatures from -10°C (coldest/blue) to 50°C (warmest/red).

What This Actually Means

This vertical line shows us the **starting voltage** of the battery at different temperatures **before any discharge has occurred**. Essentially, we're seeing:

1. Cold batteries (-10°C, blue at top) start at a higher voltage (around 3.9V)
2. As temperature increases, the starting voltage decreases
3. The warmest batteries (50°C, red) start at a lower voltage (around 3.1-3.2V)

Making Sense of This for Non-Engineers

Think of this like measuring the water pressure in hoses at different temperatures before turning on the water. We've measured the pressure (voltage), but we haven't actually let any water flow (discharge) yet.

This is useful information, but it's only showing us the starting point. It's like seeing how high different runners position themselves in starting blocks before a race begins, without seeing the actual race.

What's Missing?

A typical discharge graph would show lines extending to the right, displaying how voltage drops as the battery discharges (similar to how water pressure might drop as a tank empties). Here, we only see the starting point.

Why Temperature Affects Starting Voltage

Temperature affects the chemical reactions inside batteries:

- **Cold batteries** have slower internal chemical reactions, which creates a higher initial voltage reading (though they typically have worse performance overall)
- **Warm batteries** have more active internal chemistry, resulting in lower initial voltage but often better actual performance

This is similar to how cold honey appears thicker and sits higher in a container, while warm honey flows more easily but settles to a lower level.

Engineering Insights

From an engineering perspective, this plot reveals several important phenomena:

1. **Temperature-dependent open circuit voltage (OCV):** The data confirms that